

MathDevMCP

A Group Copilot Platform for Mathematical Development,
Documentation, and Code–Document Consistency

Chak Wong and collaborators

April 23, 2026

Contents

I	Motivation and Vision	1
1	Executive Summary	2
2	Problem Statement	4
2.1	Why existing tools are not enough	5
2.2	Project objective	5
3	Product Vision and Scope	6
3.1	Vision	6
3.2	Scope for version 1	7
3.3	Design principles	8
II	Architecture and Workflows	9
4	System Architecture	10
4.1	Layer 1: Python implementation substrate	10
4.2	Layer 2: MCP server facade	11
4.3	Layer 3: Workflow layer	11
4.4	Claude Code integration	12
4.5	Security and reliability guardrails	12
5	Core Workflows	13
5.1	Monograph and long-form documentation support	13
5.2	Code–document consistency review	13
5.3	Documentation auditing	14
5.4	Mathematically informed code generation	14
5.5	Proposal drafting and review under deadline pressure	14
5.6	Tool matrix for the group’s hardest problems	14
5.7	Recommended workflow pattern	15
5.8	Context extraction and seeded mismatch benchmarks	16

6	Group Use Cases	18
6.1	DSGE development	18
6.2	Macrofinance and empirical research documentation	18
6.3	Large monographs and appendices	19
6.4	Team workflows	19
6.5	Training and adoption implications	19
III	Evaluation and Delivery	20
7	Evaluation Plan	21
7.1	Evaluation philosophy	21
7.2	Benchmark classes	21
7.2.1	Retrieval benchmark	21
7.2.2	Verification benchmark	22
7.2.3	Repair-guidance benchmark	22
7.2.4	Derivation-backed claim benchmark	23
7.3	Evaluation modes	23
7.4	Domain-specific test criteria	23
7.4.1	Monograph support	23
7.4.2	Code–document consistency	24
7.4.3	Code generation	24
7.4.4	Documentation auditing	24
7.4.5	Team usability	24
7.5	Recommended acceptance threshold	25
7.6	Implementation test plan	25
8	Rollout, Risks, and Adoption	26
8.1	Rollout stages	26
8.2	Adoption model	27
8.3	Key project risks	27
8.4	Governance and review	27
9	Conclusion	28
	Appendices	30
A	Detailed Test Criteria	30
A.1	Monograph support tests	30
A.2	Code–document consistency tests	30

A.3	Code generation tests	31
A.4	Documentation auditing tests	31
A.5	Team adoption tests	32
B	Work Packages and Delivery Plan	33
C	Risk Register	35
C.1	Seeded mismatch fixture tests	36
C.2	Context extraction	36

Part I

Motivation and Vision

Chapter 1

Executive Summary

MathDevMCP is proposed as an internal mathematical development copilot platform for groups that build mathematically intensive software and must simultaneously produce large-scale technical documentation, proposals, and monographs under severe time pressure. The immediate motivation comes from recurring experience in DSGE development, macrofinance research, and adjacent quantitative workflows where the same team must maintain consistency across code, equations, assumptions, algorithm descriptions, and long-form exposition. Existing LLM-assisted editors have already proven useful for drafting and local editing, but they remain unreliable in the exact areas that matter most for this group: long-range dependency tracking, cross-file mathematical consistency, derivation checking, code–document traceability, and disciplined auditing of claims.

The proposal recommends building MathDevMCP as a small but rigorous internal platform rather than as a generic chatbot enhancement. The core design combines three elements: a Python implementation layer for extraction, indexing, verification, and consistency logic; a thin MCP server that exposes these capabilities to Claude Code and related tooling; and a workflow layer focused on the actual group tasks of monograph drafting, proposal writing, code–document consistency review, documentation auditing, and mathematically informed code generation. This approach is intentionally conservative. It does not assume autonomous theorem proving, fully automatic proof repair, or end-to-end generation of correct monographs. Instead, it aims to build a trustworthy system that makes mathematically heavy work faster, more reproducible, and less error-prone.

The expected value for the group is broad. For long monographs and proposals, MathDevMCP should improve retrieval, grounding, and revision consistency across hundreds of pages. For mathematical codebases, it should improve the ability to connect equations and assumptions in documents to concrete implementations and test artifacts. For auditing and review, it should support symbolic verification, numeric falsification, gap localization, and consistency checks across code and prose. For code generation, it should constrain generation against mathematical specifications and benchmark the results against executable acceptance criteria. The intent is not merely to generate nicer

text, but to reduce the incidence of mathematically confident but incorrect outputs. This design choice follows the broader view that reliable mathematical software work depends on numerical stability, explicit assumptions, and disciplined evidence trails rather than prose fluency alone [Higham, 2002, MacKay, 2003].

The proposal therefore frames MathDevMCP as a group productivity and reliability investment. Its success will be evaluated not by prose quality alone, but by measurable improvements in retrieval accuracy, code–document consistency detection, audit verdict accuracy, code generation correctness, drafting throughput, and reduction in hallucinated or unsupported mathematical claims.

Recommendation. Develop MathDevMCP as a proposal-first and benchmark-driven internal platform. Prioritize document indexing, code–document traceability, symbolic and numeric checks, and structured audit workflows before attempting ambitious autonomous generation features.

Chapter 2

Problem Statement

The group’s current workflow already demonstrates that modern coding agents and drafting assistants can materially reduce the effort of writing technical documentation. However, mathematically dense projects expose sharp limitations. The documentation corpus can easily run to 500–600 pages across chapters, appendices, technical notes, and research proposals, while the codebase simultaneously evolves through solver revisions, algorithmic experiments, and implementation hardening. In such settings, correctness depends not on any single paragraph or equation, but on long-range consistency among assumptions, notation, derivations, data structures, dimensions, transforms, numerical approximations, and algorithmic descriptions.

This creates a recurring mismatch between the strengths of current AI-assisted tools and the group’s actual needs. Present tools can help with local drafting, rephrasing, and summarization, but they struggle with multi-hop mathematical dependencies across distant files and document sections. They can restate derivations fluently without reliably checking them. They can generate code that appears aligned with a mathematical description while quietly dropping assumptions, terms, or shape constraints. They can summarize code into documentation that sounds plausible but no longer matches the implementation after a refactor. Under a short deadline, these failures become especially dangerous because they are easy to miss during rapid drafting cycles.

Several concrete problem classes motivate the project:

1. **Long-range document dependency failures.** Definitions, symbols, and assumptions introduced early in a monograph are often used much later in proofs, algorithm boxes, and appendices. Current tools lose track of these dependencies and may create notation drift or inconsistent restatements.
2. **Code–document drift.** Mathematical software frequently evolves faster than associated exposition. A proposal may describe an older algorithm variant, a chapter may mention a Jacobian term that has been removed from code, or a code path may no longer implement the assumptions described in the text.

3. **Weak mathematical verification.** A large model can produce confident expository text without proving symbolic identities, checking dimensions, or finding counterexamples. This is especially problematic for transport maps, perturbation algebra, adjoint systems, and other domains where small sign or transformation errors matter.
4. **Unstructured auditing workflows.** Existing review usually depends on manual reading, ad hoc scripts, and memory of prior incidents. There is insufficient infrastructure for repeatable claim verification, seeded inconsistency detection, or benchmarked audit quality.
5. **Deadline compression.** The group sometimes needs to produce long, mathematically coherent documentation in three to four weeks. In such windows, even strong manual workflows become fragile because retrieval, checking, and revision are too expensive.

These challenges imply that the project should not be framed merely as a writing assistant. The actual need is for a trustworthy mathematical development environment that links documents, code, and math-checking tools. Such an environment must support both human drafting and machine-assisted verification. It must provide structured retrieval over code and LaTeX, external symbolic and numeric reasoning backends, and group-facing workflows for consistency checking and auditing.

2.1 Why existing tools are not enough

General-purpose editors with embedded language models help at the paragraph level, but they do not provide the structured tool contracts needed for high-stakes mathematical development. They usually lack an explicit layer for theorem/equation indexing, do not expose domain-specific verification backends as stable tools, and do not maintain a persistent bridge between the document corpus and implementation artifacts. Without such structure, the model must repeatedly rediscover context and improvise its own checking strategy, which is inefficient and fragile.

2.2 Project objective

The objective of MathDevMCP is to reduce these failure modes by creating a common platform for retrieval, verification, consistency checking, and workflow orchestration. The platform should support large-document authoring and mathematical software development as one integrated process rather than as separate activities.

Chapter 3

Product Vision and Scope

MathDevMCP is envisioned as an internal copilot platform for mathematical development. Its purpose is to help researchers and developers work effectively across long technical documents, symbolic derivations, numerical code, and implementation-heavy research workflows. The platform should make Claude Code and related agent tools materially more reliable in the specific settings that matter for the group: DSGE development, macrofinance research, technical monographs, project proposals, and mathematically structured codebases.

3.1 Vision

The vision is not to create a generic AI writing system. The platform should instead provide a grounded, tool-mediated environment in which drafting and coding are continuously supported by retrieval, verification, and consistency checks. In the target workflow, an agent should be able to locate the relevant theorem or code path, extract the precise local context, invoke symbolic or numeric checks through a stable interface, and produce a recommendation that is explicitly tied to evidence.

At a practical level, MathDevMCP should enable the following:

- drafting long technical sections from indexed local sources rather than from free-form memory;
- deriving and checking intermediate mathematical steps in project notation before the agent makes categorical claims;
- verifying symbolic identities, intermediate derivation steps, and change-of-variable relationships with external algebra systems;
- detecting drift between code and documentation;

- auditing mathematical claims with repeatable verdicts and measurable evidence quality;
- generating code from mathematical specifications subject to explicit acceptance tests;
- supporting both expert users in Claude Code and teammates who only need batch reports or prebuilt workflows.

3.2 Scope for version 1

Version 1 should focus on a disciplined, high-value subset of capabilities.

In scope

- a. LaTeX parsing, indexing, and retrieval for chapters, labels, equations, theorem-like environments, references, and local context.
- b. Code indexing and linkable search over mathematical implementations, tests, and supporting scripts.
- c. Symbolic checks via SymPy and SymEngine, with SageMath and Maxima as richer or independent fallback backends.
- d. Numeric falsification and sanity checks via mpmath or related Python tooling.
- e. Constraint and consistency checks for suitably encodable cases via Z3.
- f. MCP exposure of these capabilities to Claude Code through stable, structured tools.
- g. Audit-oriented composite workflows such as claim verification, gap localization, counterexample search, and code–document consistency review.

Out of scope for version 1

- a. general theorem proving over arbitrary monograph content;
- b. automatic translation of unrestricted prose into formal logic;
- c. fully autonomous production of correct long-form chapters without human review;
- d. universal symbolic support for every mathematical domain equally well;
- e. large-scale distributed infrastructure or heavyweight platform engineering before local workflows have proven value.

3.3 Design principles

The project should follow a few explicit principles.

1. **Small, structured tool surface.** A few well-defined tools are preferable to many overlapping tool endpoints.
2. **Benchmark-driven development.** Capabilities should be justified by measurable improvement on representative tasks, not by intuition alone.
3. **Evidence before prose.** The platform should prefer workflows that generate traceable evidence, such as an extracted equation, symbolic simplification result, or numeric counterexample.
4. **Human-centered review.** The goal is to reduce pain and increase reliability, not to remove expert oversight.
5. **Group usability.** The system must support both interactive and batch use so that the whole group benefits.

Part II

Architecture and Workflows

Chapter 4

System Architecture

MathDevMCP should be implemented as a layered system whose main purpose is to give Claude Code and related workflows reliable access to document-aware and math-aware tools. The recommended architecture has three layers: an implementation substrate in Python, a thin MCP server facade, and a workflow layer that exposes task-oriented capabilities to users and agents.

4.1 Layer 1: Python implementation substrate

The base layer should be a local Python toolkit responsible for parsing, indexing, verification, consistency analysis, and report generation. This layer should contain the real logic of the system because it is the easiest part to test, extend, and reuse outside interactive agent sessions.

Core subcomponents should include:

1. **Document extraction and indexing.** Parse LaTeX sources, collect theorem-like environments, labels, equation blocks, references, and section structure, and store them in normalized index artifacts.
2. **Code inventory and traceability.** Build searchable inventories of functions, classes, configuration points, tests, and implementation modules relevant to mathematical workflows.
3. **Verification backends.** Provide wrappers for symbolic algebra, numeric probing, and logical consistency checking.
4. **Audit and reporting logic.** Implement task-oriented routines for claim verification, consistency checks, and benchmark scoring.

This layer should rely on local tools already available to the group: SymPy, SymEngine, SageMath, Maxima, mpmath, and Z3. The implementation should prefer lightweight on-

disk caches such as JSON and optionally SQLite for larger indexes, rather than introducing heavyweight infrastructure too early.

4.2 Layer 2: MCP server facade

The second layer should expose a carefully chosen subset of the implementation toolkit as MCP tools. Claude Code works best when tools have clear schemas and narrow semantics. Therefore, MathDevMCP should not start by exposing unrestricted shell access or large numbers of backend-specific entrypoints. Instead, it should provide structured tools such as:

- `build_document_index`
- `search_document_and_code`
- `extract_latex_context`
- `verify_identity`
- `find_numeric_counterexample`
- `check_logical_consistency`
- `audit_claim`
- `check_code_doc_consistency`

These tools should hide most backend details while still allowing backend preference or fallback selection. For example, a single identity-verification tool can default to SymPy, optionally escalate to SageMath or Maxima, and record which backend produced the result. This reduces cognitive load for the agent and makes benchmarking easier.

4.3 Layer 3: Workflow layer

The final layer should organize the tools into actual group workflows. The system should support three user surfaces:

1. **Claude Code integration.** Interactive use for drafting, auditing, code reading, and consistency checks.
2. **CLI scripts.** Repeatable commands for local and batch use, suitable for users who prefer shell workflows or want reproducible reports.
3. **Batch reports.** Chapter-level or repository-level audit reports for teammates who do not need interactive tooling.

4.4 Claude Code integration

Project-local MCP configuration should make MathDevMCP tools available only in repositories where they are relevant. Claude Code should remain the planner and synthesizer, while MathDevMCP provides retrieval, verification, and audit operations with stable contracts. This division of responsibility is important: the agent decides what to inspect and how to summarize results, while the MCP system produces structured evidence.

4.5 Security and reliability guardrails

The platform should treat external math tools as evidence engines, not as unquestionable truth oracles. Each tool invocation should record inputs, backend used, outputs, and failure conditions. Timeouts, normalization of outputs, and clear inconclusive states should be standard. The system should prefer explicit statuses such as *proved*, *disproved*, *inconclusive*, and *not encodable* rather than forcing a binary verdict.

Chapter 5

Core Workflows

MathDevMCP should be judged by whether it improves the group’s real workflows rather than by whether it produces attractive isolated outputs. This chapter defines the main workflows the platform must support.

5.1 Monograph and long-form documentation support

The first workflow is the drafting and revision of long monographs, technical notes, and project proposals. In this workflow, the user should be able to search for a theorem, proposition, label, or definition; retrieve the surrounding context; compare current text with implementation details; and draft or revise new material while remaining grounded in local evidence. The platform should make it easier to maintain notation consistency, track references, and avoid silently contradicting earlier assumptions.

5.2 Code–document consistency review

A central workflow is checking whether code and documentation remain aligned. The platform should support both directions:

- from documentation to code: does the implementation match the described objective, transformation, dimension, algorithm step, or assumption?
- from code to documentation: does the text accurately describe what the implementation actually does after recent changes?

This workflow should produce localized discrepancies rather than vague warnings. Examples include mismatched state dimensions, omitted Jacobian terms, stale algorithm names, inconsistent parameter definitions, or divergence between documented assumptions and runtime behavior.

5.3 Documentation auditing

The audit workflow should verify claims, localize proof gaps, search for counterexamples, and classify findings with explicit evidence. The goal is not just to say that something is wrong, but to say *where* it is wrong and *why*. A useful audit should distinguish among correct claims, false claims, missing assumptions, incomplete derivations, and claims that cannot be verified from local context.

5.4 Mathematically informed code generation

MathDevMCP should also support code generation from mathematical specifications, but under stricter controls than generic agent-assisted generation. The workflow should begin from a structured mathematical description or extracted formula set, generate code in a bounded scope, and then validate the result against executable acceptance checks. For mathematical code, verification should include dimension consistency, sign conventions, basic symbolic equivalence where possible, and scenario-level behavior checks. This is especially valuable for transformations, likelihood terms, linear algebra kernels, and diagnostic routines.

5.5 Proposal drafting and review under deadline pressure

The same platform should support short-cycle proposal work. In that setting, the system should help build drafts from source material, summarize prior chapters or code modules, detect contradictions, and provide review checklists. Time-to-draft matters, but unsupported claims and stale descriptions remain costly. Therefore, proposal workflows should integrate retrieval and quick consistency checks by default.

5.6 Tool matrix for the group’s hardest problems

The most difficult workflows currently faced by the group can be mapped directly to tool classes. This mapping should guide both implementation and evaluation.

Problem	Best tool classes	Why they matter
---------	-------------------	-----------------

Very long documents and many chapters	LaTeX parsing and indexing, theorem/equation inventory, label dependency graph, MCP search and context extraction	The main bottleneck is structured retrieval over long-range dependencies, not prose generation alone
Code–document consistency	Unified code and document index, traceability checker, mismatch localizer, seeded inconsistency benchmarks	The group needs localized evidence that equations, assumptions, algorithm descriptions, and implementations still match
Derivation auditing and math-aware typo detection	Symbolic checks, numeric falsification, proof-step extraction, notation drift checker, counterexample search	Many dangerous failures are mathematical near-typos, missing assumptions, or incorrect local derivation steps
AI implementation ignores the document and improvises	Document-grounded code generation, specification extraction, acceptance-test harnesses, code–spec consistency checks	The remedy is to force the generation workflow to start from retrieved specifications and validate outputs before trusting them
Difficulty reading dense mathematical exposition	Claim-context extraction, symbol expansion, equation dependency graph, local derivation-step verification	Users need tools that isolate dense local context and explain or verify it incrementally

5.7 Recommended workflow pattern

Across all workflows, the recommended pattern is:

1. retrieve the relevant document and code context;
2. extract the exact local claim, equation, or implementation region;
3. invoke the lightest appropriate verification tool;
4. escalate to richer backends only when needed;
5. summarize findings with explicit evidence and uncertainty;

6. if drafting or generating code, run acceptance checks before treating the output as trustworthy.

This pattern keeps the platform focused on evidence-based assistance rather than unconstrained generation.

Version 1

- LaTeX indexing and search
- context extraction for sections, theorem-like environments, and equations
- unified code and document search
- first-line symbolic and numeric verification wrappers
- MCP exposure for retrieval and verification tools

Version 2

- code–document consistency checks with localized mismatch reports
- audit-oriented workflows for claim verification, gap localization, and counterexample search
- richer backend fallback with SageMath and Maxima

Version 3

- document-grounded code generation
- acceptance-test harnesses derived from mathematical specifications
- team-facing batch reports and broader adoption tooling

5.8 Context extraction and seeded mismatch benchmarks

The implementation roadmap should now include two concrete early capabilities beyond plain indexing. First, the platform should support extraction of local LaTeX context around a label or structural block so that users and agents can inspect a theorem, equation, or section together with the surrounding lines. This addresses the recurring need to read dense mathematical material locally rather than as an undifferentiated whole. Second, the project should include seeded code–document mismatch fixtures in the benchmark

suite. These fixtures should deliberately introduce inconsistencies between a document statement and a small code artifact so that MathDevMCP can be tested on its ability to detect and localize mismatches before it is trusted on real project material.

These capabilities are useful because they turn abstract goals into testable implementation milestones. Context extraction supports reading, auditing, and grounded drafting. Seeded mismatch fixtures support regression testing of code–document consistency logic and reduce the risk that the project overestimates its own usefulness.

Chapter 6

Group Use Cases

The project is motivated by workflows shared across the group rather than by the needs of a single user. The group’s research and development tasks combine mathematically dense code, long technical documentation, and rapid iteration under publication or proposal deadlines. MathDevMCP should therefore be designed as a shared internal platform.

6.1 DSGE development

In DSGE development, model derivations, perturbation systems, filtering algorithms, transformations, priors, and estimation procedures all require tight alignment between equations and implementation. Typical failure cases include incorrect dimensions, missing terms in Jacobians, stale assumptions in the documentation, and algorithm descriptions that lag behind code changes. MathDevMCP should help the team trace these dependencies across code and text, and provide audit support when model or solver changes propagate through the documentation.

6.2 Macrofinance and empirical research documentation

Macrofinance work often requires long-form explanation of models, identification assumptions, empirical strategies, and numerical methods. Such projects are especially sensitive to inconsistencies in notation, conditioning assumptions, and the mapping between the paper’s mathematical objects and the actual code. The platform should help maintain coherence across research proposals, appendices, replication documentation, and implementation notes.

6.3 Large monographs and appendices

For 500–600 page monographs or technical manuals, retrieval and consistency are themselves major challenges. A productive system must help users find the right theorem, equation, or appendix entry quickly, verify that revisions do not break earlier notation, and keep cross-references stable. The platform should support chapter-level and whole-document views so that both local editing and global review are feasible.

6.4 Team workflows

The platform should not assume every teammate will work interactively with Claude Code. Some users will prefer command-line scripts, while others will mainly consume generated reports. Therefore, each core capability should be implementable in a reusable internal library, surfaced through MCP for interactive users, and also callable through CLI or batch reporting layers.

6.5 Training and adoption implications

A shared platform should reduce duplicated manual effort. Once the group agrees on workflows for indexing, auditing, and consistency review, members can reuse the same procedures across projects. This should improve onboarding, reduce dependence on individual memory, and make large writing efforts less brittle. The platform will also create a common vocabulary for review: for example, whether a finding is a false statement, a proof gap, a missing assumption, or an implementation mismatch.

Part III

Evaluation and Delivery

Chapter 7

Evaluation Plan

MathDevMCP should be evaluated by whether it improves mathematically meaningful outcomes rather than merely whether it produces fluent prose. The evaluation framework should therefore measure retrieval quality, audit correctness, consistency-checking effectiveness, code generation quality, and real workflow usability.

7.1 Evaluation philosophy

The proposal recommends a benchmark-driven evaluation strategy. Each evaluation task should have explicit expected outputs, measurable scoring rules, and a clear definition of what counts as success, failure, or inconclusive evidence. Whenever possible, the platform should be tested on tasks grounded in existing documents, code, and archived review findings rather than on synthetic prompts alone.

7.2 Benchmark classes

7.2.1 Retrieval benchmark

The retrieval benchmark measures whether the system can find the correct local mathematical or implementation context from a natural-language prompt.

Representative tasks include:

- finding the proposition defining a Jacobian determinant identity;
- locating the chapter and equation where a transport map is introduced;
- finding the implementation associated with a documented algorithm step;
- retrieving the assumptions relevant to a later proof or appendix remark.

Primary metrics:

- correct file/chapter hit rate;
- correct proposition/equation/label hit rate;
- rank of first correct hit;
- time to answer;
- hallucinated reference count.

7.2.2 Verification benchmark

The verification benchmark measures whether the system can determine whether a claim is correct, false, gapful, ambiguous, or unverifiable from local context.

Representative tasks include:

- checking whether a proposition matches its proof;
- verifying a change-of-variables identity;
- determining whether a stationarity condition is sufficient as stated;
- deciding whether an implementation-level transformation matches the documented formula.

Primary metrics:

- verdict accuracy;
- evidence quality;
- symbolic derivation success;
- counterexample success when a claim is false;
- false positive and false negative rates.

7.2.3 Repair-guidance benchmark

The repair-guidance benchmark measures whether the system can identify the broken step, missing assumption, or local correction needed after a problem has been found.

Primary metrics:

- localization accuracy;
- whether the proposed repair is locally correct;
- whether the proposal reduces ambiguity rather than introducing new errors.

7.2.4 Derivation-backed claim benchmark

This benchmark measures whether the system can avoid unsupported categorical claims by producing either a local derivation in project notation or a cited equation plus a logical explanation.

Primary metrics:

- fraction of categorical claims backed by an explicit derivation or cited equation;
- correctness of the derivation step classification;
- rate of unsupported confident claims;
- usefulness of the produced derivation for human review.

7.3 Evaluation modes

The proposal recommends evaluating at least four modes:

1. Claude-only baseline;
2. Claude plus retrieval/indexing tools;
3. Claude plus math backends;
4. Claude plus full MathDevMCP stack.

For internal analysis, richer ablations can distinguish parser/index gains from symbolic, numeric, and logical gains. This will help the group avoid overbuilding underused features.

7.4 Domain-specific test criteria

7.4.1 Monograph support

Success criteria should include:

- improved retrieval of chapters, propositions, equations, definitions, and references;
- lower notation drift across long documents;
- reduced unsupported claims in drafted material;
- lower revision time for cross-chapter updates;
- improved consistency of citations and internal references.

7.4.2 Code–document consistency

Success criteria should include:

- detection of seeded mismatches between equations and implementation;
- detection of stale algorithm descriptions after code changes;
- traceability between documented assumptions and code-level artifacts;
- ability to explain mismatches precisely rather than vaguely.

7.4.3 Code generation

Success criteria should include:

- generated code passes mathematical acceptance tests;
- generated code preserves dimensions, sign conventions, and transformation structure;
- lower hallucination rate than baseline agent-assisted code generation;
- generated code is localizable to the source mathematical specification.

7.4.4 Documentation auditing

Success criteria should include:

- higher verdict accuracy on archived audited claims;
- better proof-gap localization;
- successful production of counterexamples when claims fail;
- reduced rate of confident but unsupported audit conclusions.

7.4.5 Team usability

Success criteria should include:

- shorter time-to-first-draft and time-to-reviewed-draft;
- lower manual cleanup burden;
- repeatable use by multiple group members;
- clear failure modes and escalation paths.

7.5 Recommended acceptance threshold

A feature should be considered valuable only if it improves at least one important benchmark class without materially degrading the others. In practice, the strongest near-term criterion is likely improved verdict accuracy and localization on previously audited claims, combined with lower manual effort in drafting and consistency review.

7.6 Implementation test plan

The implementation itself should be tested in the same spirit as the product. Version 1 should include automated tests for:

- LaTeX index construction on small fixture documents;
- search ranking for theorem, equation, section, and label queries;
- extraction of local context with correct file and line spans;
- CLI commands for building and querying indexes;
- symbolic and numeric wrapper smoke checks when optional math backends are installed;
- MCP tool schema validation once the MCP facade is introduced.

Version 2 should add seeded inconsistency tests in which a document fixture and code fixture intentionally disagree about an equation, dimension, algorithm name, or assumption. A code–document consistency feature should not be considered usable until it can detect these seeded mismatches and report their locations.

Version 3 should add code-generation acceptance tests. Each generated implementation should be evaluated against examples or formulas extracted from the source document, not only against generic style checks.

Chapter 8

Rollout, Risks, and Adoption

The project should be delivered in stages. A staged approach is important because the largest technical risk is not that the underlying tools are unavailable, but that the platform becomes too broad, too noisy, or too difficult to trust before the highest-value workflows are stabilized.

8.1 Rollout stages

Stage 1: indexing and retrieval MVP

Implement LaTeX extraction, chapter/equation/theorem inventories, code inventory, and unified search over document and code artifacts. Deliver MCP tools for search and context extraction, together with a small benchmark suite.

Stage 2: verification backends

Add symbolic identity checking, numeric falsification, and basic logical consistency checks through a small number of structured tools. Integrate backend selection and failure reporting.

Stage 3: audit and consistency workflows

Implement composite tools for claim verification, counterexample search, proof-gap localization, and code–document consistency review. Add seeded inconsistency tests and report generation.

Stage 4: code generation support and wider adoption

Add mathematically constrained code-generation workflows and acceptance-test harnesses. Expand CLI and batch reporting surfaces so the platform supports the full group.

8.2 Adoption model

The platform should be introduced as a shared internal service with three access modes: interactive Claude Code workflows, CLI scripts, and batch reports. This allows gradual adoption without requiring all teammates to change their habits at once. Documentation should include examples, benchmark tasks, and a small set of recommended review procedures.

8.3 Key project risks

Risk 1: Overbuilding. A large general-purpose math agent platform could absorb significant effort before proving value. The mitigation is to keep the first release small and benchmark-driven.

Risk 2: Tool noise and false confidence. Symbolic or logical tools can produce outputs that are technically correct but irrelevant to the real mathematical issue. Mitigation requires structured schemas, clear statuses, and explicit evidence recording.

Risk 3: Fragile encodings. Some claims do not translate naturally into symbolic or solver-ready form. The platform must support “inconclusive” and “not encodable” outcomes instead of overclaiming.

Risk 4: Workflow friction. If the system is only usable by one expert, group-level adoption will fail. The platform therefore needs CLI and report surfaces in addition to MCP integration.

Risk 5: Benchmark mismatch. A benchmark that rewards polished explanations rather than mathematical correctness could push development in the wrong direction. The mitigation is to score retrieval, verdict quality, localization, and executable consistency outcomes.

8.4 Governance and review

The proposal recommends treating MathDevMCP as an internal research engineering project with explicit milestone reviews. Each stage should end with a small benchmark report, examples from real group workflows, and a decision on whether to expand the platform or simplify it.

Chapter 9

Conclusion

MathDevMCP is proposed as a practical response to a recurring group-wide problem: mathematically intensive software and mathematically intensive documentation are developed together, but existing AI-assisted workflows do not reliably maintain the consistency, verification discipline, and long-range context needed for high-quality outcomes under tight deadlines. The group does not merely need better prose assistance. It needs a trustworthy environment for retrieval, verification, consistency checking, auditing, and controlled code generation.

The proposal recommends a disciplined path. Build a local Python toolkit that performs the real work of parsing, indexing, verification, and reporting. Expose a carefully selected subset of that functionality through an MCP server so Claude Code can use it interactively. Support the whole group through parallel CLI and batch-report interfaces. Evaluate every major capability using benchmarks that reflect the group’s actual workflows: monograph support, code–document consistency, code generation correctness, documentation auditing, and usability under deadline pressure.

If executed in stages, MathDevMCP can become a durable internal capability rather than a one-off experiment. It can reduce duplicated review effort, lower the rate of unsupported or inconsistent mathematical claims, improve code–document traceability, and make long-form technical writing less brittle. For a group working on DSGE development, macrofinance, and similar mathematically structured projects, that would represent a meaningful improvement in both productivity and reliability.

Appendices

Appendix A

Detailed Test Criteria

This appendix provides explicit test criteria for the first major evaluation domains. The intent is to make MathDevMCP measurable and falsifiable.

A.1 Monograph support tests

Test area	Task	Pass criteria
Retrieval	Find the exact proposition, definition, equation, or appendix entry for a natural-language query	Correct file and local label found with high top- k recall and low hallucinated references
Notation consistency	Update a section that reuses notation from distant chapters	No introduced symbol drift, inconsistent reuse, or broken cross-reference
Draft grounding	Draft a section from indexed local sources	Claims are linked to retrieved evidence and unsupported additions are flagged
Revision support	Apply a change to notation or assumptions and propagate consequences	Affected chapters and references are correctly identified and revision checklist is generated

A.2 Code–document consistency tests

Test area	Task	Pass criteria
Equation–code alignment	Compare a documented formula with its implementation	Correctly identify match or mismatch and localize the relevant code path

Assumption consistency	Check whether stated assumptions are enforced or relied upon in code	Correct trace from assumption to implementation/test artifact
Algorithm drift	Detect when text describes an obsolete algorithm variant	Seeded drift is detected with high precision and actionable localization
Dimension consistency	Compare documented dimensions with runtime structures	Shape mismatch or agreement is stated correctly and supported by evidence

A.3 Code generation tests

Test area	Task	Pass criteria
Specification fidelity	Generate code from a mathematical specification	Generated code preserves the required structure, variables, and conventions
Executable correctness	Run unit and scenario checks derived from the specification	Generated code passes acceptance checks without hidden manual repair
Traceability	Explain where the generated code came from	Output is tied back to equations, assumptions, and source snippets
Hallucination control	Inspect for invented routines or unsupported simplifications	Low hallucination rate relative to baseline generation workflows

A.4 Documentation auditing tests

Test area	Task	Pass criteria
Verdict accuracy	Classify a claim as correct, incorrect, gapful, ambiguous, or unverifiable	Classification matches gold or reviewer consensus
Gap localization	Identify the broken step or missing assumption	Localized finding points to the right proof segment, equation, or code path
Counterexample generation	Produce a contradiction when a claim is false	Counterexample is valid, relevant, and reproducible

Unsupported claim detection	Flag claims not justified by local evidence	Reduced rate of confident unsupported conclusions
--------------------------------	--	--

A.5 Team adoption tests

- A new teammate can use a documented workflow without bespoke coaching.
- The same task produces similar outputs across users because tools and reports are structured.
- Time-to-first-draft and time-to-reviewed-draft decrease on representative tasks.
- Failure modes are understandable and do not silently masquerade as success.

Appendix B

Work Packages and Delivery Plan

This appendix breaks the project into concrete work packages.

WP1: Document and code indexing

- Build LaTeX extraction for chapters, labels, theorem-like blocks, equations, and references.
- Build code inventory for functions, classes, tests, and relevant symbols.
- Produce local caches and unified search.
- Deliver first retrieval benchmark.

WP2: Verification backends

- Add SymPy and SymEngine first-line symbolic checks.
- Add mpmath numeric probing.
- Add SageMath and Maxima fallback wrappers.
- Add Z3-based consistency checks for encodable cases.
- Normalize outputs into structured result records.

WP3: MCP interface

- Implement a local Python MCP server.
- Expose a small tool surface with structured schemas.

- Add project-local configuration for Claude Code.
- Create examples and smoke tests.

WP4: Audit and consistency workflows

- Implement claim auditing and gap localization.
- Implement code–document consistency checks.
- Produce batch reports and benchmark outputs.

WP5: Code generation support

- Define mathematically constrained generation tasks.
- Add acceptance-test harnesses based on formulas and examples.
- Benchmark generated code against baseline workflows.

WP6: Adoption and documentation

- Write user-facing workflow guides.
- Create sample benchmark tasks and example reports.
- Train group members on interactive and batch usage patterns.

Appendix C

Risk Register

Risk		Likelihood	Impact	Mitigation
Overly	broad	Medium	High	Keep the tool surface small, use staged delivery, and require benchmark evidence before expansion
Noisy	symbolic	Medium	Medium	Normalize outputs, record backend provenance, and support inconclusive states
Fragile solver	encodings	Medium	Medium	Restrict logical checks to encodable cases and explicitly label not-encodable tasks
Weak group	adoption	Medium	High	Support Claude Code, CLI, and batch reports; document workflows with examples
Benchmark	drift	Low	High	Use real group tasks and archived audit findings instead of only synthetic examples
Maintenance	burden	Medium	Medium	Build reusable Python internals and avoid unnecessary infrastructure

False confidence in generated code	Medium	High	Require executable acceptance tests and traceability to the mathematical specification
------------------------------------	--------	------	--

C.1 Seeded mismatch fixture tests

A particularly important implementation test class is the seeded mismatch fixture. In these tests, a small LaTeX fragment and a small code fragment intentionally disagree about a required mathematical term, algorithm component, dimension, or correction term. The consistency layer should then report a mismatch and identify the missing terms or relevant location. These fixtures are not substitutes for real-project evaluation, but they are an effective regression guard while the platform is still young.

C.2 Context extraction

MathDevMCP should expose local context extraction as a first-class capability. A useful assistant must not only find the right theorem or equation label, but also present the nearby text needed to understand the claim, assumptions, and local proof structure. Early implementations should therefore support extracting a few lines around a labeled theorem, equation, or section boundary.

Bibliography

Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2002.

David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.